Part 1: Repair the Ticketing System

Objective

Use AI-assisted debugging tools to investigate and repair the intentionally broken ticketing application. The goal is to restore the application to a reliable, usable state while understanding why each defect occurred.

The project uses:

- FastAPI for the REST API
- DuckDB for persistence
- Pydantic for validation and data models
- NiceGUI for the web interface

Your Task

Create a Github Public Repo and Upload the code.

Find and fix the errors in the `app/` package. The application contains several different kinds of defects, including:

- A syntax error that prevents the application from starting
- Database schema and SQL errors
- Incorrect database result mapping
- Broken filtering and search logic
- API response and HTTP status errors
- Input validation problems
- Off-by-one ID errors
- UI form wiring mistakes
- UI styling and visibility problems

Do not assume that the first error reported is the only problem. Once one issue is fixed, run the application again and continue investigating the next failure.

Completion Criteria

You are finished when:

- The server starts without traceback errors.
- The API checks above produce the expected results.
- The UI workflows work correctly in a browser.

- Automated tests pass.
- The test suite includes regression coverage for the repaired behaviors.
- You can explain the root cause and fix for each category of error.
- The working-tree diff contains only relevant application and test changes.

Deliverable:

- A short debugging summary listing the errors found, their root causes, and the tools or checks used to verify the fixes.
- Error Free / Fixed Code

Part 2: Refactoring Task

Objective

Refactor the ticketing system to improve readability, maintainability, and testability while preserving its current observable behavior. The result should be easier for another engineer to extend without changing the HTTP API, MCP interface, database behavior, or NiceGUI workflow.

Non-goals

- Do not add new product features.
- Do not change API routes, HTTP status codes, request or response models, MCP tool names, MCP payloads, or error messages.
- Do not change database schema, migrations, column order, timestamp semantics, filtering behavior, sorting, or seed data.
- Do not change NiceGUI layout, notifications, event ordering, or callback behavior.
- Do not replace DuckDB, FastAPI, Pydantic, MCP, or NiceGUI with another library.
- Do not perform broad formatting, dependency, or unrelated bug-fix changes.

Deliverables

- Refactoring Rules in Markdown to be added into the repo
- Refactored Code

Complete Criteria

The work is complete when:

- all existing tests pass;
- all newly added focused tests pass;
- the implementation preserves the public and persistence contracts listed above;
- API and MCP error translation is no longer unnecessarily duplicated;
- repository SQL construction remains parameterized and field names remain explicitly controlled;
- no unrelated files or generated Sphinx output are changed;
- the final diff is small enough to review by behavior and purpose;
- the participant can explain why each extracted helper exists and what behavior it preserves.

At minimum, run the focused tests for the files being changed first. Before submission, run the full command above. If formatting, linting, or type-checking tools are available in the project environment, run them and report the results. Do not substitute a failing or mismatched environment without documenting the limitation.

Submission Notes

1. The Github Repo Link that contains the repaired source code.

Do not simply remove functionality to make an error disappear. Preserve the intended ticketing behavior and fix the code at the point where the incorrect behavior is introduced.